



Build a Juicy Spin Wheel

Create a reward wheel players love to tap — data-driven, portable, and packed with polish.

ENGINE
Unity

ARCHITECTURE
State Machine

FOCUS
Polish + Reuse

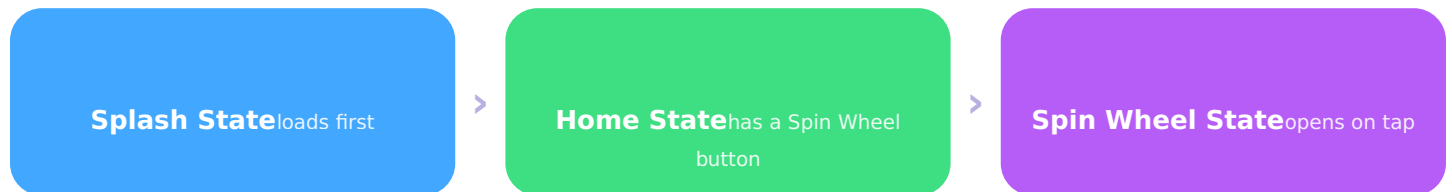
◆ The Mission

You're building a reusable **Spin Wheel** for a mobile game — the kind players tap to win coins. It must be **fully data-driven**: a designer who doesn't code should be able to edit one config file and completely change the wheel. Just as important, your spin wheel must be built as a **self-contained, drop-in container** — something you could lift out and drop into any other project that uses the same game architecture, with no rewrites.



◆ Game Flow & Architecture

Use the **State Machine architecture provided in the project**. Wire the flow exactly like this:



Splash shows first → transitions to **Home**. On Home there is a **Spin Wheel button**; pressing it enters the **Spin Wheel State**. Keep each state clean and self-contained so states can be added or removed without breaking the others.

◆ Requirements

1 Dynamic slice generation

Build the slices at runtime from the config — the slice count comes from the number of rewards, always evenly divided and correctly drawn.

2 JSON-driven configuration

Read everything from the provided **data.json**. Editing the JSON changes the wheel — no code edits.

3 Configurable probabilities

Each slice has a probability from the config. Common rewards land often, rare ones land rarely.

4 Controlled stop index

Stop on any requested slice (e.g. 0–7 for 8 slices) and still look natural and seamless.

5 Smart, reusable code

Clean OOP. Package the wheel as a portable container that drops into any project with similar architecture.

6 Configurable spin system

Expose spin duration, rotations/cycles, stop index, animation curve, and a test mode for repeated spins.

◆ Provided Data — data.json

coins is the player's starting balance. Each entry in rewards is one slice, in order. Award coins based on the winning slice's multiplier.

```
{
  "coins": 300,
  "rewards": [
    { "multiplier": 5, "probability": 0.4, "color": "#FFCE36" },
    { "multiplier": 10, "probability": 0.2, "color": "#F9536E" },
    { "multiplier": 20, "probability": 0.15, "color": "#FF9C43" },
    { "multiplier": 50, "probability": 0.1, "color": "#FF75CB" },
    { "multiplier": 80, "probability": 0.07, "color": "#3EDF82" },
    { "multiplier": 100, "probability": 0.05, "color": "#41A7FF" },
    { "multiplier": 200, "probability": 0.02, "color": "#5E6DFF" },
    { "multiplier": 500, "probability": 0.01, "color": "#B75DF7" }
  ]
}
```

multiplier — reward shown & awarded **probability** — chance to win (sums to 1) **color** — slice fill (hex)

Your system must still work if this file is edited — more or fewer rewards, different multipliers, probabilities, or colors.

◆ Pro Tips — Make It Shine

Chase the Juice

- ◆ **Make it as juicy as possible.** A spin wheel lives and dies on *feel* — squash & stretch, easing, a satisfying tick as the pointer passes each slice, a punchy stop, screen shake, confetti or particles on a win, and a reward pop-up that celebrates.
- ◆ **Polish and finishing matter most.** A simple wheel that feels amazing beats a feature-heavy one that feels flat. Sound, animation curves, and micro-interactions are where the marks are.
- ◆ **Code smartly.** Build the wheel as a **self-contained, portable container** — decoupled from the rest of the game — so it can be dropped into any project with a similar architecture without rewrites.
- ◆ **Lean on the state machine.** Keep Splash, Home, and Spin Wheel as clean, independent states with clear transitions.
- ◆ **Keep data and logic separate.** The JSON (or config asset) decides *what*; your code decides *how*.

◆ Deliverables

- 📁 Complete Unity project (exclude the **Library** folder)
- 📁 Your **data.json** and any config assets used
- 📄 Short README: how to run it & how a designer reconfigures the wheel
- 📺 A 15–30s screen recording / GIF of the wheel spinning

Assets: art is already provided in the project. You may also **generate your own assets using AI** if you want to add or restyle anything.

Build the core solidly first, then make it shine.
Good luck & have fun!